Experiment No. 5

Title

Fetch API Call to Public API and Display JSON Data Dynamically

Aim

To retrieve and display data from a public API using the JavaScript Fetch API and process JSON responses dynamically.

Software Required

- Text Editor (Notepad / VS Code)
- Web Browser (Google Chrome / Mozilla Firefox)

Theory

JavaScript Fetch API

The Fetch API is a modern JavaScript interface used to make HTTP requests to servers. It allows web applications to retrieve data asynchronously from external sources such as public APIs. Fetch API returns a Promise that resolves to a response object, which can be processed further.

JSON (JavaScript Object Notation)

JSON is a lightweight, text-based data format used for storing and exchanging data between a client and a server. It represents data in the form of key-value pairs and arrays, making it easy to read and process.

What JSON Can Do

- Represent structured data using objects and arrays
- Transfer data between client and server
- Support nested and hierarchical data
- Enable fast data exchange due to lightweight format
- Work across programming languages
- Enable dynamic content generation in web applications
- Serve as the standard data format for REST APIs

Processing JSON Responses Dynamically

Processing JSON responses dynamically means that the data received from the API is:

- Converted into JavaScript objects
- Accessed programmatically
- Displayed on the webpage without hardcoding
- Rendered at runtime without page reload

This approach improves flexibility, performance, and user experience.

Public APIs

A public API is an API that is freely accessible and provides data in JSON format for learning and development purposes. Full list can be accessed here:https://github.com/public-apis/public-apis?tab=readme-ov-file

Examples of Public APIs

- **JSONPlaceholder** – Users, Posts, Comments
  https://jsonplaceholder.typicode.com

- **Fake Store API** – Products & pricing
  https://fakestoreapi.com

- **REST Countries API** – Country information
  https://restcountries.com

- **Random User API** – User profiles
  https://randomuser.me/api

- **Dog CEO API** – Dog images
  https://dog.ceo/api

Commands Used to Process and Present JSON Data :

Command	**Purpose**
fetch()	Retrieve data from public API
response.json()	Convert response to JavaScript object
forEach()	Iterate through JSON array
slice()	Limit number of records
createElement()	Create HTML elements dynamically
innerHTML	Insert JSON data into HTML
appendChild()	Display data on webpage
async / await	Handle asynchronous operations

try–catch	Error handling

1. Create a new folder for the experiment.
2. Create three files: index.html, style.css, and script.js.
3. Write HTML code to structure the webpage.
4. Apply CSS styling and animations for better presentation.
5. Use Fetch API to retrieve JSON data from a public API.
6. Convert and process the JSON response dynamically.
7. Display the data on the webpage.
8. Run the program in a web browser and observe the output.

Program Code :

- **Exp5.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fetch API Demo</title>
  <link rel="stylesheet" href="Exp5.css">
</head>

<body>

<header>
  <h1>Fetch API Demo</h1>
  <p>Fetching Anime Data & Images Dynamically</p>
</header>
```

```
<button id="loadBtn">Load Top Anime</button>
<button id="sortBtn">Sort</button>
<select id="genre" class="sorting">
   <option >Select</option>
   <option value="1">Adventure</option>
   <option value="2">Action</option>
   <option value="3">Sci-fi</option>
</select>

<div id="output"></div>

<script src="Exp5.js"></script>
</body>
</html>
```

- **Exp5.css :**

```
body {
   font-family: Arial, sans-serif;
   color: white;
   background-color: #000000;
   padding: 30px;
   text-align: center;
}

header {
   margin-bottom: 20px;
}

button {
   padding: 12px 20px;
   background-color: #7c3aed;
```

```
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    font-size: 16px;
    transition: transform 0.2s ease, background-color 0.3s ease;
}

button:hover {
    background-color: #5b21b6;
    transform: scale(1.05);
}

@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(15px) rotate(0deg);

    }
    to {
        opacity: 1;
        transform: translateY(0) rotate(1turn);
    }
}

.sorting {
    background-color: #6d26df;
    color: #ffffff;
    border-radius: 5px;
    padding: 5px;
}
```

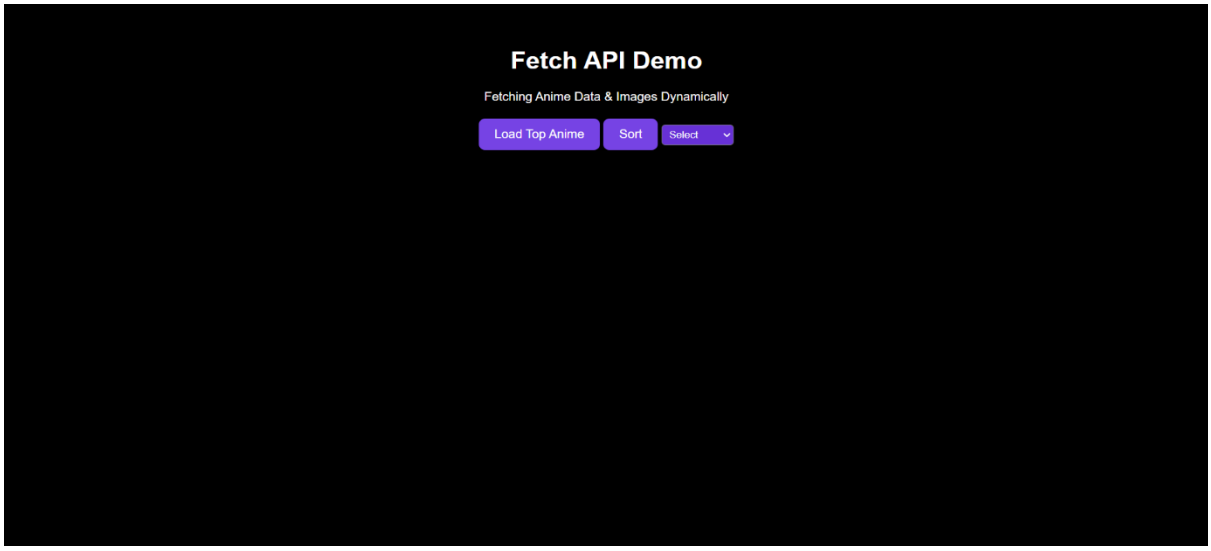
```
.card {
   background-color: #5d00ff;
   width: 280px;
   margin: 15px;
   padding: 15px;
   border-radius: 10px;
   box-shadow: 0 6px 12px rgb(0, 238, 255);
   animation: fadeIn 1.5s ease-in-out;
   transition: transform 0.3s ease-in-out;
}

.card:hover {
   box-shadow: 0 6px 12px rgb(255, 0, 0);
   transform: scale(1.02);
   background-color: #2b2b2b;
}

.card img {
   width: 100%;
   border-radius: 8px;
}

.container {
   display: flex;
   flex-wrap: wrap;
   justify-content: center;
}
```

- **Exp5.js :**

```
const loadBtn = document.getElementById("loadBtn");
const output = document.getElementById("output");
const sortBtn = document.getElementById("sortBtn");
let genre = document.getElementById("genre");

loadBtn.addEventListener("click", async () => {
  try {
    output.innerHTML = "Loading anime data...";

    const response = await fetch("https://api.jikan.moe/v4/top/anime");

    if (!response.ok) {
      throw new Error("Network error");
    }

    const result = await response.json();
    const animeList = result.data.slice(0, 20);

    output.innerHTML = `<div class="container"></div>`;
    const container = document.querySelector(".container");

    animeList.forEach(anime => {
      const card = document.createElement("div");
      card.className = "card";

      const genreNames = anime.genres
      .map(g => g.name)
      .join(", ");

      card.innerHTML = `
```

```
            <img src="${anime.images.jpg.image_url}" alt="${anime.title}">
            <h3>${anime.title}</h3>
            <p><strong>Rating :</strong> ${anime.score}</p>
            <p><strong>Episodes :</strong> ${anime.episodes}</p>
            <p><strong>Genre :</strong> ${genreNames}</p>
          `;

          container.appendChild(card);
        });

    } catch (error) {
        output.innerHTML = "Failed to load anime data.";
        console.error(error);
    }
});

sortBtn.addEventListener("click", async () => {
    try {
        output.innerHTML = "Loading anime by genre...";

        const genreId = genre.value;
        const response = await fetch(
            `https://api.jikan.moe/v4/anime?genres=${genreId}&order_by=score&sort=desc`);

        if (!response.ok) {
            throw new Error("Network error");
        }

        const result = await response.json();
        const animeList = result.data.slice(0, 12);
```

```
    output.innerHTML = `<div class="container"></div>`;
    const container = document.querySelector(".container");

    animeList.forEach(anime => {
      const card = document.createElement("div");
      card.className = "card";

      const genreName = anime.genres
      .map(g => g.name)
      .join(", ");

      card.innerHTML = `
        <img src="${anime.images.jpg.image_url}" alt="${anime.title}">
        <h3>${anime.title}</h3>
        <p><strong>Rating :</strong> ${anime.score}</p>
        <p><strong>Episodes :</strong> ${anime.episodes}</p>
        <p><strong>Genre :</strong> ${genreName}</p>
      `;

      container.appendChild(card);
    });

  } catch (error) {
    output.innerHTML = "Failed to load anime data.";
    console.error(error);
  }
});
```

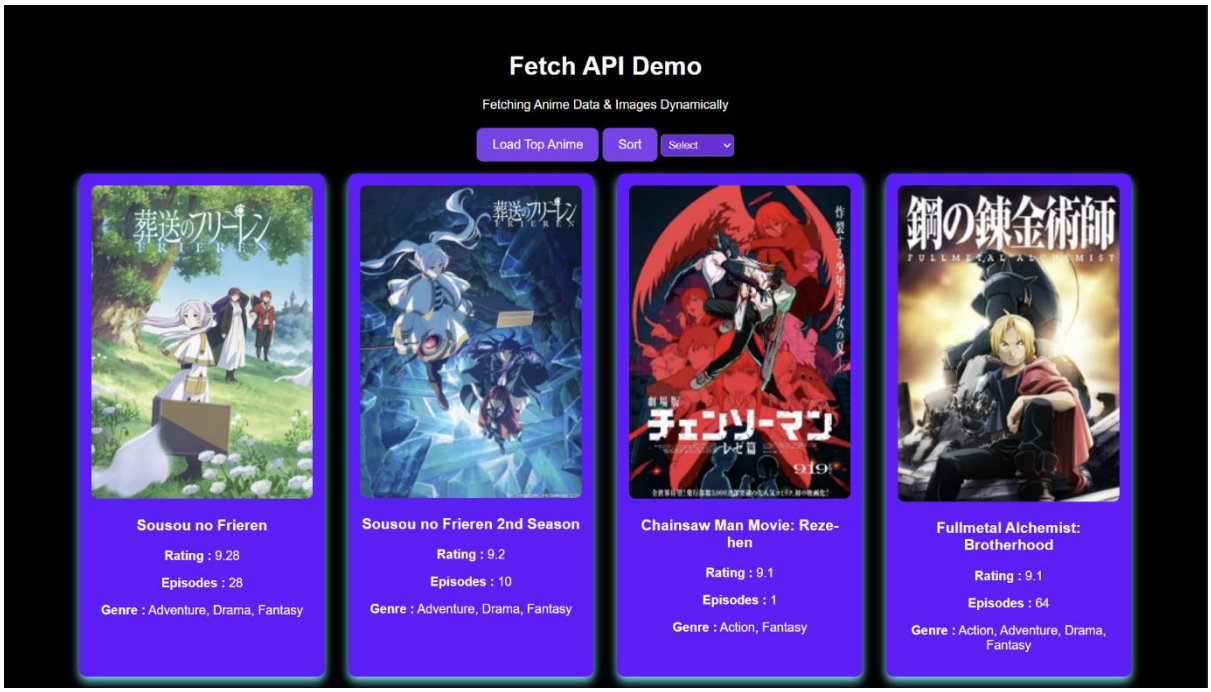


Student Tasks :

Task 1 : Display additional anime details such as synopsis or episode count.

Code :

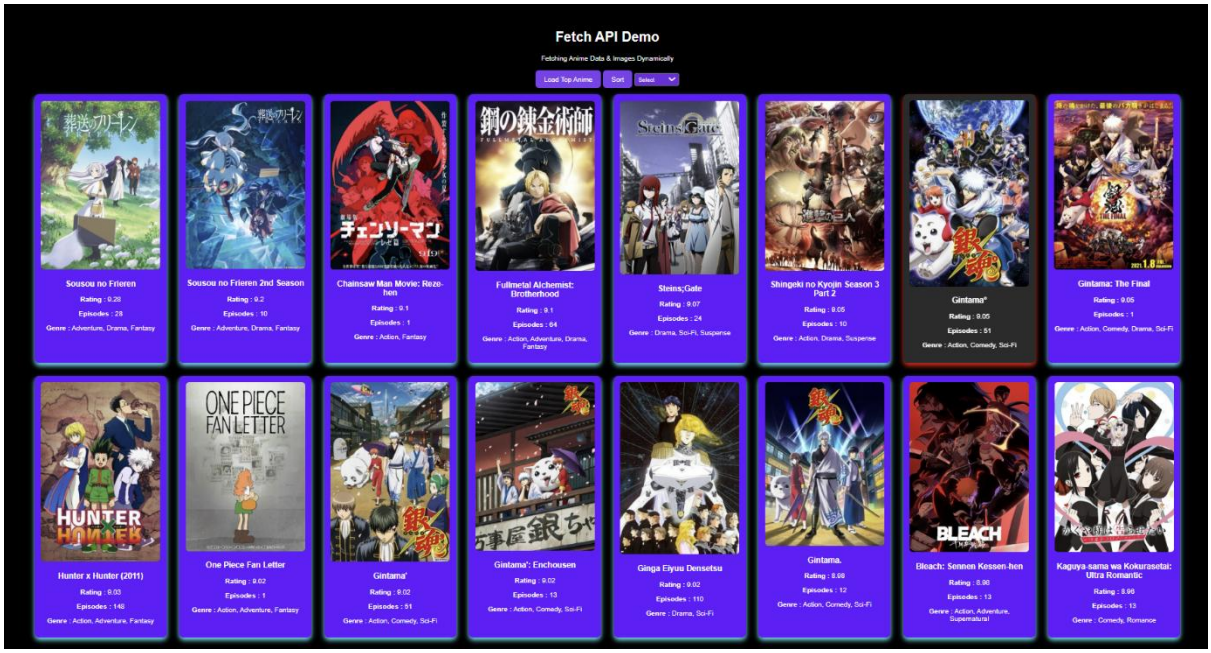
```
card.innerHTML = `
  <img src="${anime.images.jpg.image_url}" alt="${anime.title}">
  <h3>${anime.title}</h3>
  <p><strong>Rating :</strong> ${anime.score}</p>
  <p><strong>Episodes :</strong> ${anime.episodes}</p>
  <p><strong>Genre :</strong> ${genreNames}</p>
`;
```



Task 2 : Increase or decrease the number of anime cards displayed.

Code :

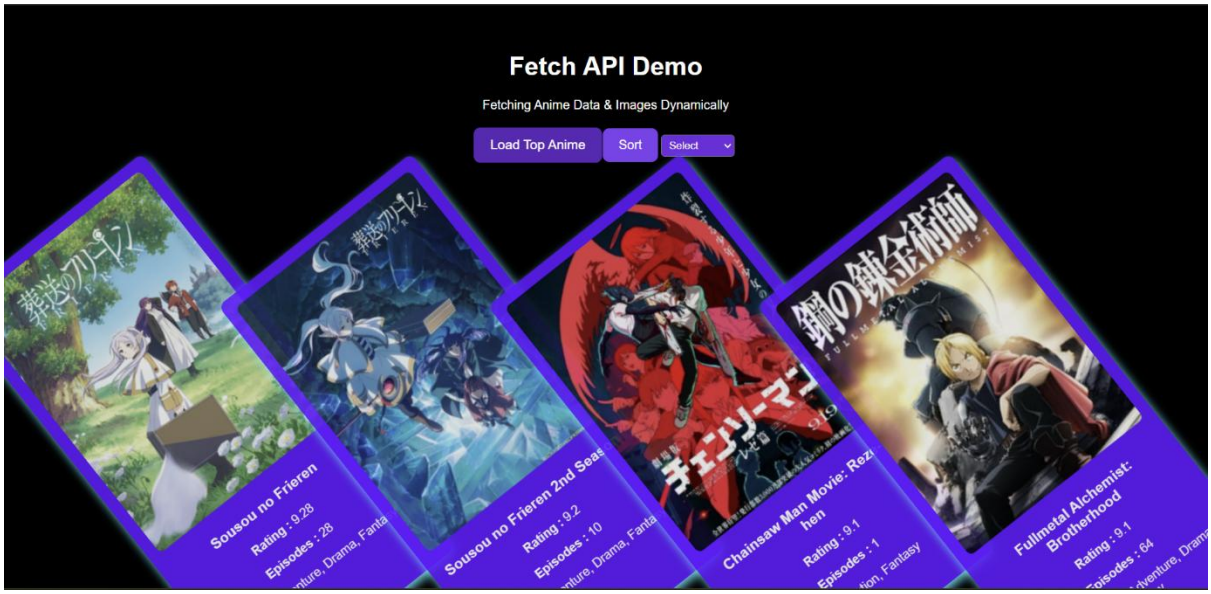
const animeList = result.data.slice(0, 20);



Task 3 : Apply different animations to anime cards by doing changes to the css file.

Code :

```
@keyframes fadeIn {
   from {
      opacity: 0;
      transform: translateY(15px) rotate(0deg);

   }
   to {
      opacity: 1;
      transform: translateY(0) rotate(1turn);
   }
}
```

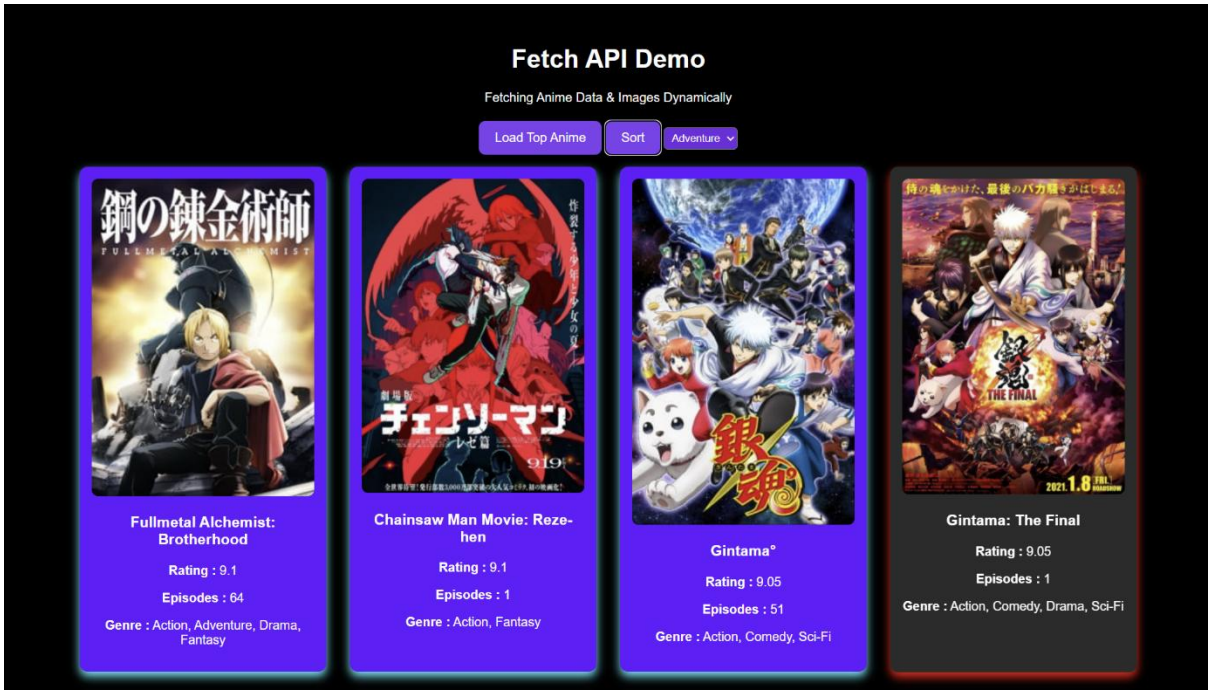


Task 3 : Modify the program to fetch anime by genre.

Code :

```
sortBtn.addEventListener("click", async () => {
  try {
    output.innerHTML = "Loading anime by genre...";
    const genreId = genre.value;
    const response = await fetch(
      `https://api.jikan.moe/v4/anime?genres=${genreId}&order_by=score&sort=desc`);
    if (!response.ok) {
      throw new Error("Network error");
    }
    const result = await response.json();
    const animeList = result.data.slice(0, 12);
    output.innerHTML = `<div class="container"></div>`;
    const container = document.querySelector(".container");

    animeList.forEach(anime => {
      const card = document.createElement("div");
      card.className = "card";
      const genreName = anime.genres
```

```
        .map(g => g.name)
        .join(", ");
        card.innerHTML = `
          <img src="${anime.images.jpg.image_url}" alt="${anime.title}">
          <h3>${anime.title}</h3>
          <p><strong>Rating :</strong> ${anime.score}</p>
          <p><strong>Episodes :</strong> ${anime.episodes}</p>
          <p><strong>Genre :</strong> ${genreName}</p>
        `;
        container.appendChild(card);
      });
    } catch (error) {
      output.innerHTML = "Failed to load anime data.";
      console.error(error);
    }
});
```



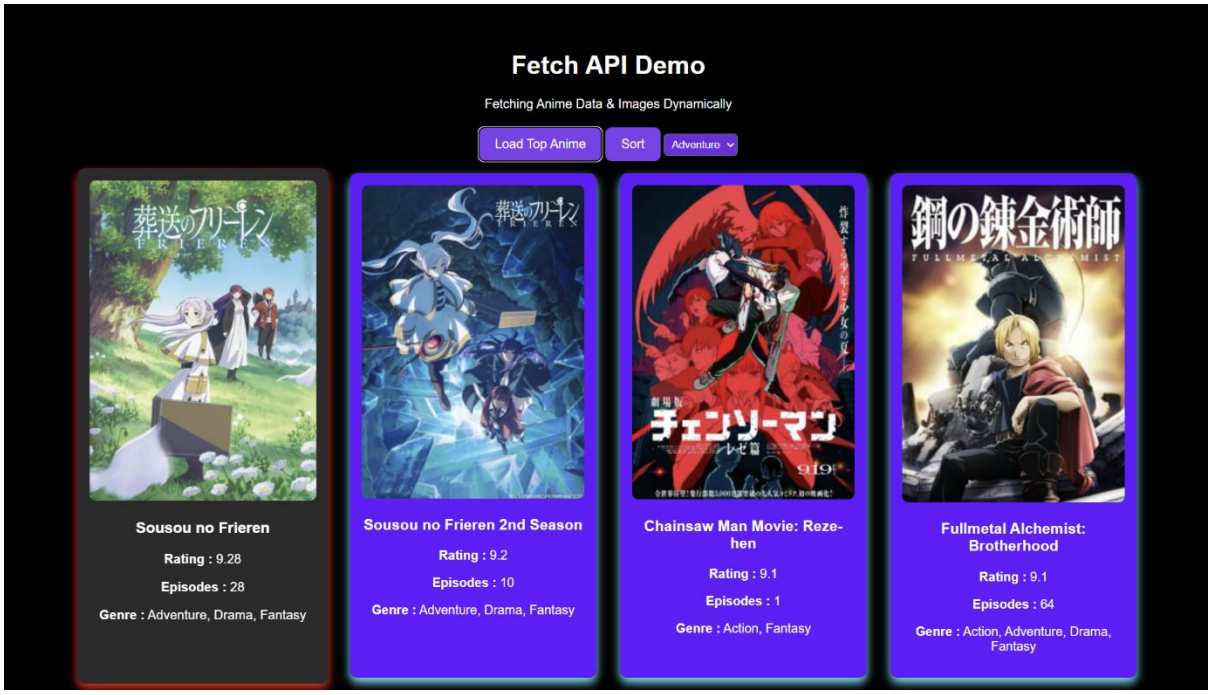
Task 4 : Convert async/await logic to .then() syntax.

Code :

```
loadBtn.addEventListener("click", () => {
  output.innerHTML = "Loading anime data...";

  fetch("https://api.jikan.moe/v4/top/anime")
    .then(response => {
      if (!response.ok) {
        throw new Error("Network error");
      }
      return response.json();
    })
    .then(result => {
      const animeList = result.data.slice(0, 20);

      output.innerHTML = `<div class="container"></div>`;
      const container = document.querySelector(".container");

      animeList.forEach(anime => {
        const card = document.createElement("div");
        card.className = "card";

        const genreNames = anime.genres
          .map(g => g.name)
          .join(", ");

        card.innerHTML = `
          <img src="${anime.images.jpg.image_url}" alt="${anime.title}">
          <h3>${anime.title}</h3>
          <p><strong>Rating :</strong> ${anime.score}</p>
          <p><strong>Episodes :</strong> ${anime.episodes}</p>
```

```
            <p><strong>Genre :</strong> ${genreNames}</p>
          `;

          container.appendChild(card);
        });
      })
      .catch(error => {
        output.innerHTML = "Failed to load anime data.";
        console.error(error);
      });
});

sortBtn.addEventListener("click", () => {
  output.innerHTML = "Loading anime by genre...";

  const genreId = genre.value;

  fetch(`https://api.jikan.moe/v4/anime?genres=${genreId}&order_by=score&sort=desc`)
    .then(response => {
      if (!response.ok) {
        throw new Error("Network error");
      }
      return response.json();
    })
    .then(result => {
      const animeList = result.data.slice(0, 12);
      output.innerHTML = `<div class="container"></div>`;
      const container = document.querySelector(".container");
      animeList.forEach(anime => {
        const card = document.createElement("div");
        card.className = "card";
```

```
        const genreName = anime.genres
          .map(g => g.name)
          .join(", ");
        card.innerHTML = `
          <img src="${anime.images.jpg.image_url}" alt="${anime.title}">
          <h3>${anime.title}</h3>
          <p><strong>Rating :</strong> ${anime.score}</p>
          <p><strong>Episodes :</strong> ${anime.episodes}</p>
          <p><strong>Genre :</strong> ${genreName}</p>
        `;
        container.appendChild(card);
      });
    })
    .catch(error => {
      output.innerHTML = "Failed to load anime data.";
      console.error(error);
    });
});
```



Task 5 : Do all the above operations on another public API of your choice.

Html and css file is the same with just some minor changes, the only major change is in the javascript file.

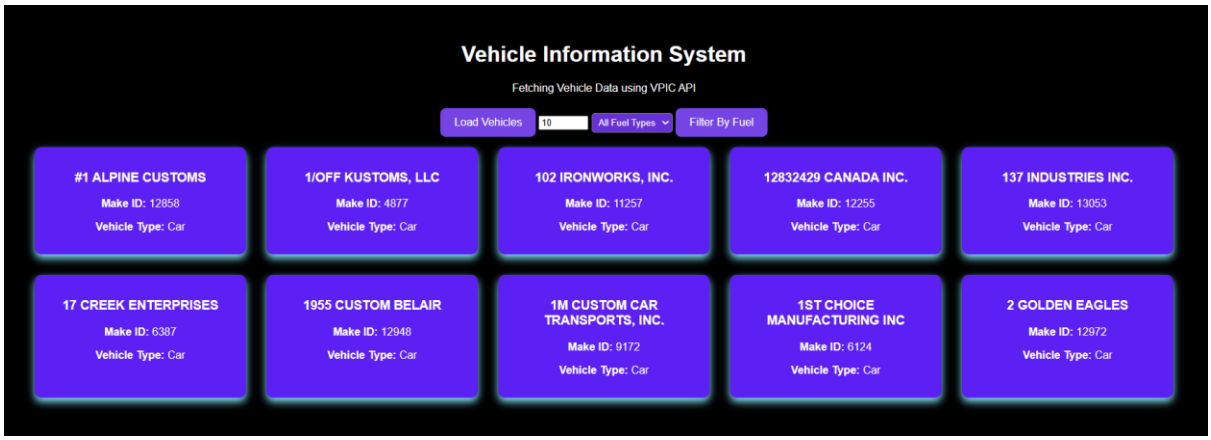
JavaScript : Exp5.1.js

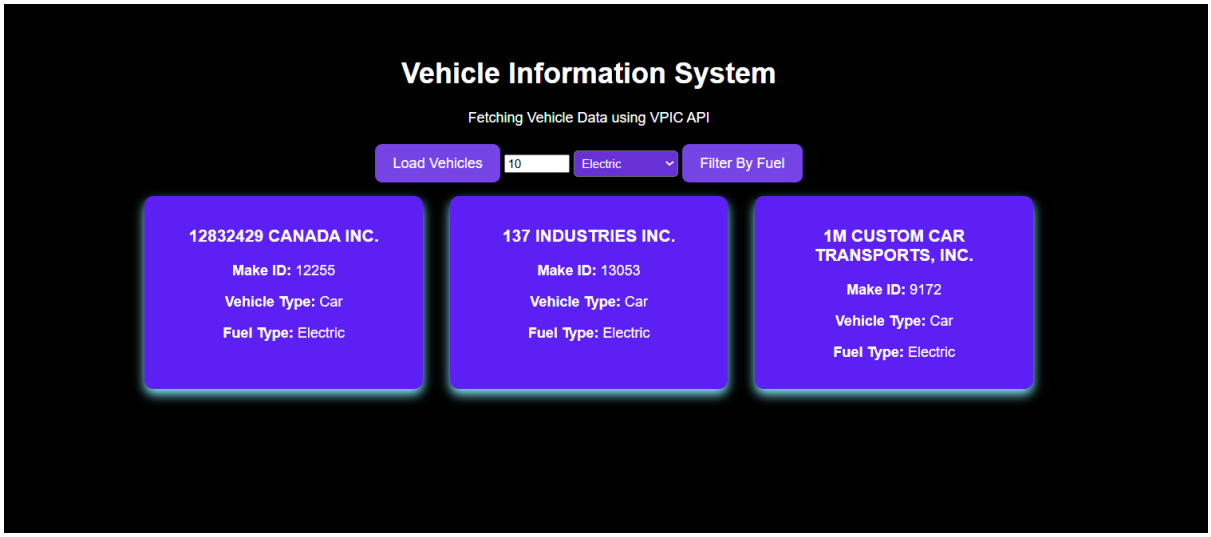
```
const loadBtn = document.getElementById("loadBtn");

const output = document.getElementById("output");

const cardCountInput = document.getElementById("cardCount");

const fuelFilter = document.getElementById("fuelFilter");

const filterBtn = document.getElementById("filterBtn");


let vehiclesData = [];


function getRandomFuel() {

  const fuels = ["Petrol", "Diesel", "Electric"];

  return fuels[Math.floor(Math.random() * fuels.length)];

}


loadBtn.addEventListener("click", function () {


  output.innerHTML = "Loading vehicle data...";


  fetch("https://vpic.nhtsa.dot.gov/api/vehicles/GetAllMakes?format=json")

    .then(response => {

      if (!response.ok) {

        throw new Error("Network error");

      }

      return response.json();

    })

    .then(result => {


      const count = parseInt(cardCountInput.value) || 10;
```

```
        vehiclesData = result.Results.slice(0, count);

        displayVehicles(vehiclesData);
    })
    .catch(error => {
        output.innerHTML = "Failed to load vehicle data.";
        console.error(error);
    });
});

filterBtn.addEventListener("click", function () {

  const selectedFuel = fuelFilter.value;

  if (selectedFuel === "all") {
    displayVehicles(vehiclesData);
    return;
  }

  const filtered = vehiclesData.filter(vehicle => vehicle.FuelType === selectedFuel);
  displayVehicles(filtered);
});

function displayVehicles(data) {

  output.innerHTML = `<div class="container"></div>`;
  const container = document.querySelector(".container");

  data.forEach(vehicle => {

    const card = document.createElement("div");
```

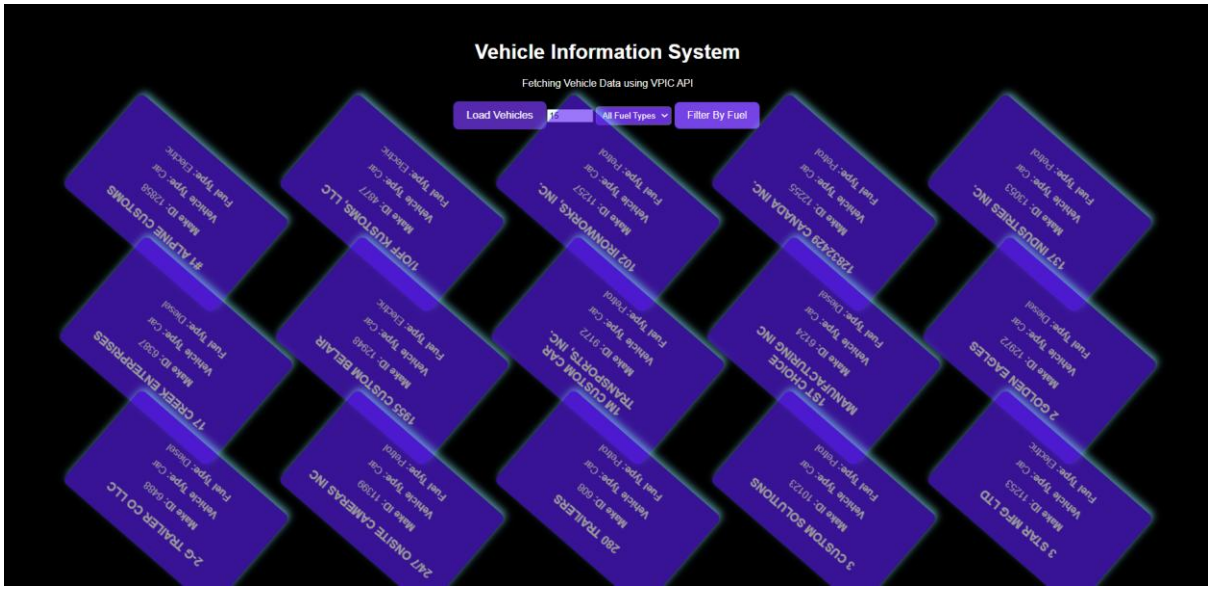
```
    card.className = "card";
    vehicle.FuelType = vehicle.FuelType || getRandomFuel();
    card.innerHTML = `
      <h3>${vehicle.Make_Name}</h3>
      <p><strong>Make ID:</strong> ${vehicle.Make_ID}</p>
      <p><strong>Vehicle Type:</strong> Car</p>
    `;

    container.appendChild(card);
  });
}
```

Output





Vehicle Information System
Fetching Vehicle Data using VPIC API
Load Vehicles
15
All Fuel Types
Filter By Fuel
#1 ALPINE CUSTOMS
Make ID: 12858
Vehicle Type: Car
Fuel Type: Petrol
1/OFF KUSTOMS, LLC
Make ID: 4877
Vehicle Type: Car
Fuel Type: Diesel
102 IRONWORKS, INC.
Make ID: 11257
Vehicle Type: Car
Fuel Type: Electric
12832429 CANADA INC.
Make ID: 12255
Vehicle Type: Car
Fuel Type: Diesel
137 INDUSTRIES INC.
Make ID: 13053
Vehicle Type: Car
Fuel Type: Diesel
17 CREEK ENTERPRISES
Make ID: 6387
Vehicle Type: Car
Fuel Type: Petrol
1955 CUSTOM BELAIR
Make ID: 12948
Vehicle Type: Car
Fuel Type: Diesel
1M CUSTOM CAR TRANSPORTS, INC.
Make ID: 9172
Vehicle Type: Car
Fuel Type: Electric
1ST CHOICE MANUFACTURING INC
Make ID: 6124
Vehicle Type: Car
Fuel Type: Diesel
2 GOLDEN EAGLES
Make ID: 12972
Vehicle Type: Car
Fuel Type: Electric
2-G TRAILER CO LLC
Make ID: 6488
Vehicle Type: Car
Fuel Type: Electric
24/7 ONSITE CAMERAS INC
Make ID: 11399
Vehicle Type: Car
Fuel Type: Electric
280 TRAILERS
Make ID: 608
Vehicle Type: Car
Fuel Type: Petrol
3 CUSTOM SOLUTIONS
Make ID: 10123
Vehicle Type: Car
Fuel Type: Diesel
3 STAR MFG LTD
Make ID: 11253
Vehicle Type: Car
Fuel Type: Diesel


Vehicle Information System
Fetching Vehicle Data using VPIC API
Load Vehicles
15
All Fuel Types
Filter By Fuel